

HOMWORK 5: REINFORCEMENT LEARNING AND UNSUPERVISED LEARNING

10-701 Introduction to Machine Learning
(PhD) (Fall 2024)

Carnegie Mellon University

<https://piazza.com/cmu/fall2024/10701/home>

OUT: November 12th, 2024*

DUE: November 26th, 2024 11:59 PM

START HERE: Instructions

- **Collaboration policy:** Collaboration on solving the homework is allowed, after you have thought about the problems on your own. It is also OK to get clarification (but not solutions) from books or online resources, again after you have thought about the problems on your own. There are two requirements: first, cite your collaborators fully and completely (e.g., “Jane explained to me what is asked in Question 2.1”). Second, write your solution *independently*: close the book and all of your notes, and send collaborators out of the room, so that the solution comes from you only. See the Academic Integrity Section in our [Course Syllabus](#) for more information:
- **Late Submission Policy:** See the late homework policy in our [Course Syllabus](#).
- **Submitting your work:** There will be two submission slots for this homework on Gradescope, the Written and the Programming:
 - **Written:** Submit your homework to the Homework 5: Written Gradescope submission slot. Please use the provided template. The best way to format your homework is by using the Latex template released in the handout and writing your solutions in Latex. However submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Each derivation/proof should be completed in the boxes provided below the question, **you should not move or change the sizes of these boxes** as Gradescope is expecting your solved homework PDF to match the template on Gradescope. If you find you need more space than the box provides you should consider cutting your solution down to its relevant parts, if you see no way to do this, please add an additional page at the end of the homework and guide us there with a ‘See page xx for the rest of the solution’.
 - **Programming:** You are also required to upload your code, which you wrote to solve the final questions of this homework, to the Programming submission slot. Your code may be run by TAs so please make sure it is in a workable state.
- Regrade requests can be made after the homework grades are released, however this gives the TA the opportunity to regrade your entire paper, meaning if additional mistakes are found then points will be deducted.

*Compiled on Tuesday 12th November, 2024 at 21:38

For multiple choice or select all that apply questions, shade in the box or circle in the template document corresponding to the correct answer(s) for each of the questions. For $\LaTeX$ users, use $\blacksquare$ and $\bullet$ for shaded boxes and circles, and don't change anything else. If an answer box is included for showing work, **you must show your work!**

1 Reinforcement [60 pts]

Consider the following fully-observable environment for reinforcement learning:

- There are k states, numbered $1, 2, \dots, k$ from left to right.
- There are two actions, L and R .
- If the state is currently $s \geq 2$, then choosing L will yield a reward of -1 and set the state to $s = 1$ (i.e., choosing L will always lead to the first state). Or, if the state is currently $s = 1$, choosing L will end the run, with a reward of 0 .
- If the state is currently $s \leq k - 1$, then choosing R will yield a reward of -1 and set the state to $s + 1$. Or, if the state is currently $s = k$, choosing R will end the run, with a reward of $k + 2$.

We would like to maximize undiscounted total reward. Maximization is well defined since the only positive rewards are when the run ends, meaning the total reward is bounded above, and since there exists at least one policy that guarantees the run will end, meaning the maximum is greater than $-\infty$.

Our policy has one parameter θ . It chooses R with probability $\sigma(\theta) = 1/(1 + \exp(-\theta))$, independent of the current state.

You may use any combination of pen, paper, coins/dice, and code to solve this problem. Whatever tools you use, please show your work and code.

1.1 Optimal policy and value

Set the number of states to be $k = 3$ for all of the questions below

1. [5 pts] What is the optimal policy for this environment? You should be able to answer this question by inspection, no need to run an RL algorithm.

2. [5 pts] Can our policy class approximate the optimal policy, and if so, how should we set θ ?

3. [5 pts] What is the value function for this optimal policy?

1.2 Simulations

1. [10 pts] Set $\theta = 0$ and simulate at least three trajectories from the above environment under our policy. Report each trajectory by listing a sequence of `(state=s1, action=a1, reward=r1)`, `(state=s2, action=a2, reward=r2)`, ... until the end of the run. For each trajectory, also report the **total reward**.

1.3 REINFORCE

1. **[15 pts]** Calculate the REINFORCE update based on the batch of trajectories that you simulated, and report the new value of θ . Use a learning rate of $\frac{1}{n}$ if you simulated n trajectories; i.e., use the average update across trajectories. Use a baseline of 0. (Recall that each REINFORCE update is a sum over time steps in a trajectory, and we are additionally averaging over a batch of trajectories.)

1.4 Baseline

1. **[15 pts]** Now calculate the REINFORCE update starting from $\theta = 0$ again, but this time with a baseline equal to the optimal value function from above. As before, answer numerically for the batch of n trajectories that you simulated, and answer qualitatively for the limit as $n \rightarrow \infty$.

2. **[5 pts]** How do these updates differ from the ones without a baseline?

2 K-Means [32 pts]

In this problem you will implement Lloyd's method for the k -means clustering problem and answer several questions about the k -means objective, Lloyd's method, and k -means++.

Recall that given a set $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ of n points in d -dimensional space, the goal of k -means clustering is to find a set of centers $c_1, \dots, c_k \in \mathbb{R}^d$ that minimize the k -means objective:

$$\sum_{j=1}^n \min_{i \in \{1, \dots, k\}} \|x_j - c_i\|^2,$$

which measures the sum of squared distances from each point x_j to its nearest center.

In class we discussed that finding the optimal centers for the k -means objective is NP-hard, which means that there is likely no algorithm that can efficiently compute the optimal centers. Instead, we often use Lloyd's method, which is a heuristic algorithm for minimizing the k -means objective that is efficient in practice and often outputs reasonably good clusterings. Lloyd's method maintains a set of centers $c_1, \dots, c_k$ and a partitioning of the data X into k clusters, $\mathcal{C}_1, \dots, \mathcal{C}_k$. The algorithm alternates between two steps: (i) improving the partitioning $\mathcal{C}_1, \dots, \mathcal{C}_k$ by reassigning each point to the cluster with the nearest center, and (ii) improving the centers $c_1, \dots, c_k$ by setting c_i to be the mean of those points in the set $\mathcal{C}_i$ for $i = 1, \dots, k$. Typically, these two steps are repeated until the clustering converges (i.e., the partitioning $\mathcal{C}_1, \dots, \mathcal{C}_k$ remains unchanged after an update). Pseudocode is given below:

1. Initialize the centers $c_1, \dots, c_k$ and the partition $\mathcal{C}_1, \dots, \mathcal{C}_k$ arbitrarily.
2. Do the following until the partitioning $\mathcal{C}_1, \dots, \mathcal{C}_k$ does not change:
 - (a) For each cluster index i , let $\mathcal{C}_i = \{x \in X : x \text{ is closer to } c_i \text{ than any other center}\}$, breaking ties arbitrarily but consistently.
 - (b) For each cluster index i , let $c_i = \frac{1}{|\mathcal{C}_i|} \sum_{x \in \mathcal{C}_i} x$.

2.1 Implementing Lloyd's Method

In this problem you will implement and experiment with Lloyd's method and the k -means++ algorithm on the two dimensional dataset shown in Figure 2.1. You will complete code in the provided Jupyter notebook.

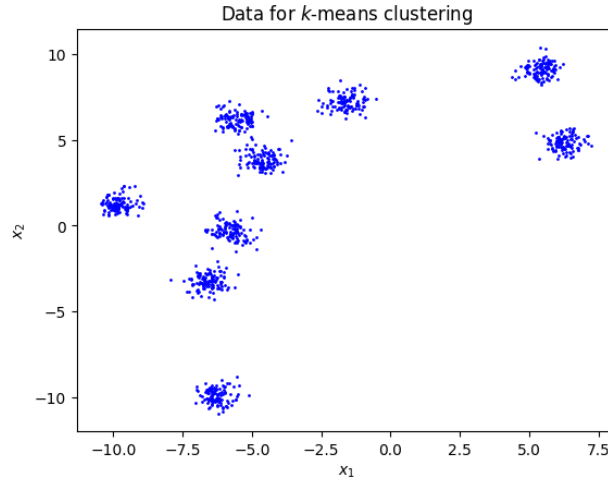


Figure 2.1: The 2D dataset X used for problem 2.

Follow the steps below to complete the programming portion of this homework:

- Complete the function `update_assignments(X, C, a)`. The input X is the $n \times d$ data matrix, C is the $k \times d$ matrix of current centers, and a is the $n \times 1$ vector of current cluster assignments. That is, $C[i, :]$ is the center for cluster i , and $a[j]$ denotes the cluster label $\{1, \dots, k\}$ for the j^{th} data point. Your function should output a new $n \times 1$ vector of cluster assignments so that each point is assigned to the cluster with the center that has the smallest Euclidean distance with the point.
- Complete the function `update_centers(X, C, a)`. The input arguments are as in part (a). Your function should output a $k \times d$ matrix C whose i^{th} row is the optimal cluster center for those points in cluster i .
- Complete the function `lloyd_iteration(X, C0)`. This function takes a data matrix X , initial centers $C0$ and runs Lloyd's method until convergence. Specifically, alternate between updating the assignments and updating the centers until the assignments stop changing. Your function should output the final $k \times d$ matrix C of centers and the final $n \times 1$ vector a of assignments.
- Complete the function `kmeans_obj(X, C, a)`. This function takes the $n \times d$ data matrix X , a $k \times d$ matrix C of centers, and an $n \times 1$ vector a of cluster assignments. Your function should output the value of the k -means objective of the provided clustering.

We provide you with a function `kmeans_cluster(X, k, init, num_restarts)` that takes an $n \times d$ data matrix X , the number k of clusters, a string `init` which must be either `'fixed'`, `'random'` or `'kmeans++'`, and a number of restarts `num_restarts`. This function runs Lloyd's method `num_restarts` times and outputs the best clustering it finds.

- When `init` is set to `'fixed'`, we initialize the centers to start at a fixed set of values. When using this you must set the initial number of clusters to be 9.
- When `init` is set to `'random'` then the centers are initialized uniformly at random.

- When `init` is set to `'kmeans++'` the centers are initialized using the D^2 weighting of k -means++.

Note that the `kmeans_cluster` function will properly work only if you complete implementing the four functions we discussed above. Use the `kmeans_cluster` function when answering the following questions.

2.2 Experiment: Testing Implementation

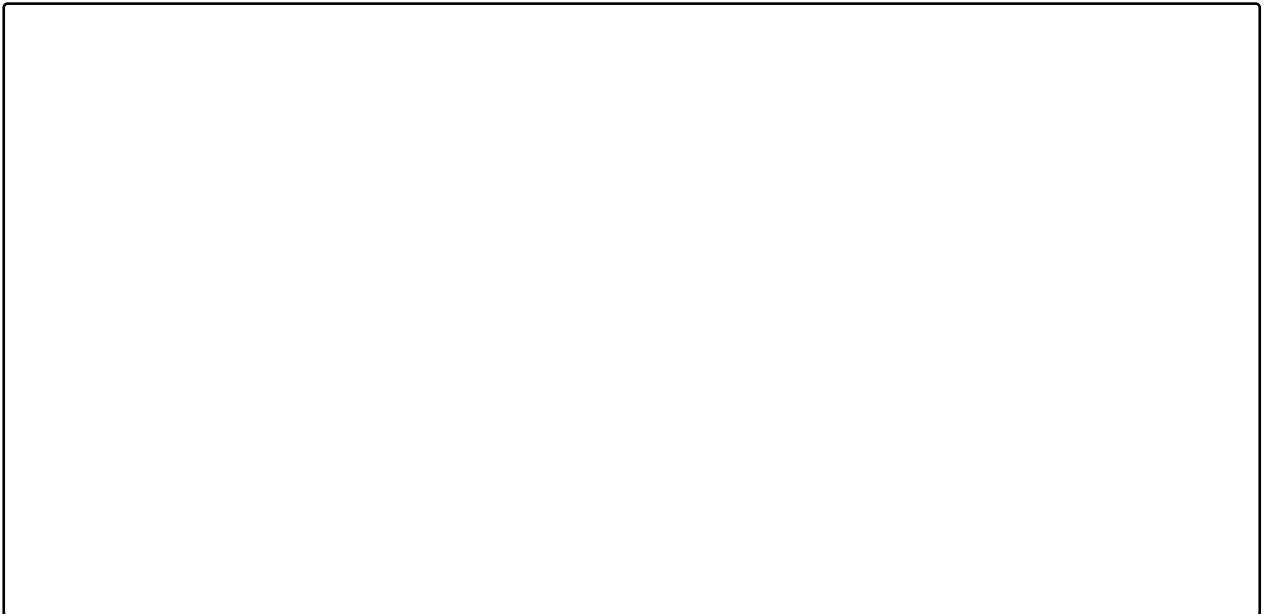
In this part of the homework we will test your implementation by asking you to calculate some values which you should generate using `init = 'fixed'` (and hence `k = 9`).

1. **[2 pts]** What is the initial value of the k -means objective function before you run any steps of Lloyd's method?

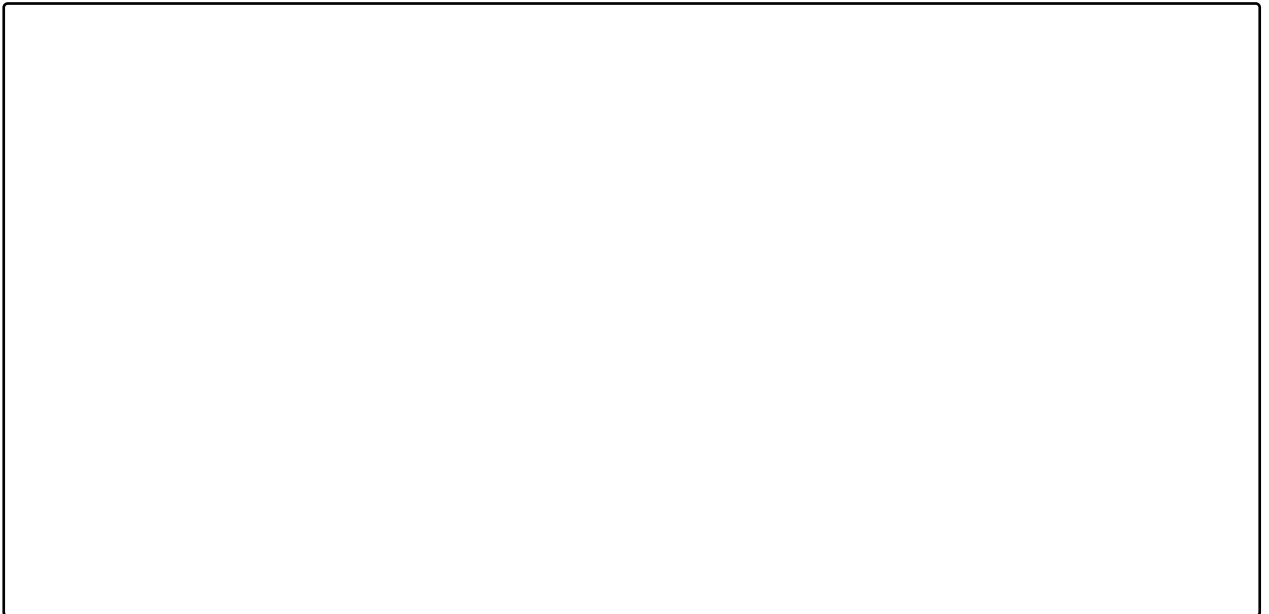
2. **[2 pts]** What is the value of the k -means objective function after 1 iteration of K-Means?

3. **[2 pts]** How many iterations until Lloyd's method converges?

4. **[5 pts]** Generate the following plot: Plot the values of the k -means objective function on the Y-axis against the number of iterations on the X-axis. Begin at iteration 1 until convergence.



5. **[5 pts]** Use the provided function `plot_kmeans_init_projs` to plot projections similar to Figure 2.1, except that the points are colored based on their class label, as well as the optimized centers, which shows the final assignments of all points.



2.3 Experiment: The effect of initialization on Lloyd's method

In this part of the homework you will investigate the effect of the center initialization on Lloyd's method. In particular, you will compare random initialization with the k -means++ initialization. Note that the provided `kmeans_cluster` function will be useful for the following questions.

1. **[4 pts]** For each value of `init` in `{'random', 'kmeans++'}`, use the provided function `plot_kmeans_init_projs` to plot projections similar to Figure 2.1, except that the points are colored based on their class label, as well as the optimized centers. Complete the code in the notebook. Use the following hyperparameters to generate both the plots and the corresponding final k-means objective values.

- `k = 9` (number of clusters)
- `num_restarts = 10` (number of restarts)

Include the plots and the objective values below.

2. **[6 pts]** For each value of `init` in `{'random', 'kmeans++'}`, report the mean and standard deviation of the k -means objective value returned by `kmeans_cluster(X, 9, init, 1)` over 1000 runs. Note that `num_restarts = 1`. There should be a clear gap in performance between the two initialization methods. Complete the code in the notebook and report the values below.

3. **[6 pts]** Intuitively, a good center initialization for the 2D dataset shown in Figure 2.1 is when we pick one center from each of the 9 natural Gaussian clusters. When two centers land in the same Gaussian cluster, they will split that cluster in half and leave only 7 centers to cover the remaining 8 Gaussian clusters. Explain in 1–2 sentences why using k -means++ initialization is more likely to choose one sample from each cluster than random initialization.

3 k -Means Extra Credit Questions [38 Extra Credit]

3.1 k -Means on the Real Line [EXTRA CREDIT: 23 Points]

In this problem you will show that the k-means objective:

$$\sum_{j=1}^n \min_{i \in \{1, \dots, k\}} \|x_j - c_i\|^2 \quad (3.1)$$

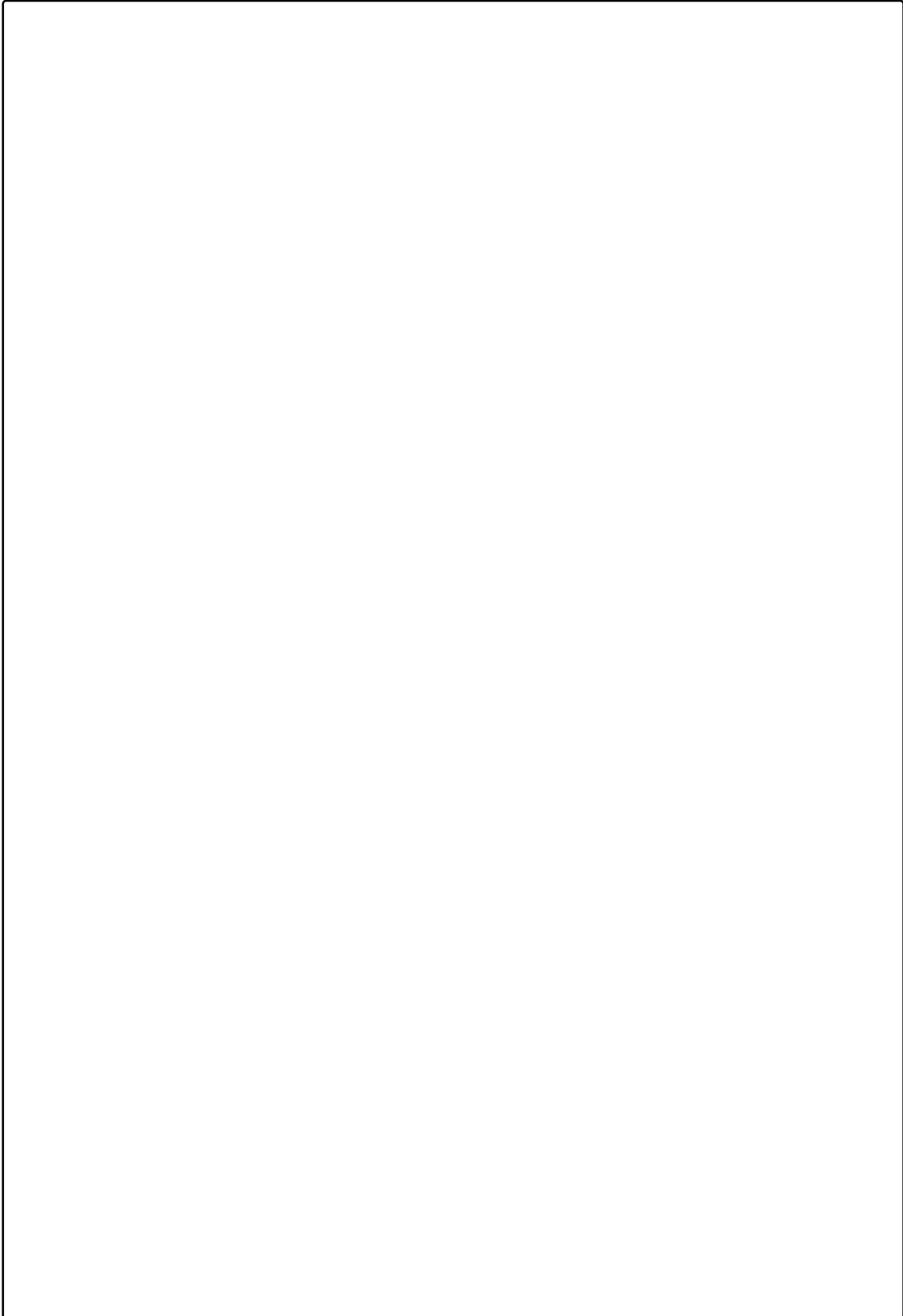
can be minimized in polynomial time when the data points are single dimensional ($d = 1$), despite the fact that in general finding the optimal centers is NP-hard.

1. **[Extra credit: 5 pts]** Consider the case where $k = 3$ and we have 4 data points $x_1 = 1, x_2 = 3, x_3 = 6, x_4 = 7$. What is the optimal clustering for this data? What is the corresponding value of the objective 3.1?

2. **[Extra credit: 5 pts]** One might be tempted to think that Lloyd's method is guaranteed to converge to the global minimum when $d = 1$. Show that there exists a suboptimal cluster assignment for the data in the previous question (Q1) that Lloyd's algorithm will not be able to improve (to get full credit, you need to show the assignment, show why it is suboptimal and explain why it will not be improved).

3. **[Extra credit: 5 pts]** Assume we sort our data points such that $x_1 \leq x_2 \leq \dots \leq x_n$. Prove that an optimal cluster assignment has the property that each cluster corresponds to some interval of points. That is, for each cluster j , there exists i_1 and i_2 such that the cluster consists of $\{x_{i_1}, x_{i_1+1}, \dots, x_{i_2}\}$

4. **[Extra credit: 8 pts]** Develop an $O(kn^2)$ dynamic programming algorithm for single dimensional k -means. (Hint: from the previous question (3), what we need to optimize are $k - 1$ cluster boundaries where the i^{th} boundary marks the largest point in the i^{th} cluster.)



3.2 k -Means Can be Kernelized Too! [EXTRA CREDIT: 15 Points]

k -means with Euclidean distance metric assumes that each pair of clusters is linearly separable. This may not be the case. A classical example is where we have two clusters corresponding to data points on two concentric circles in the $\mathbb{R}^2$ plane. We have seen that we can use kernels to obtain a non-linear version of an algorithm that is linear by nature and k -means is no exception. Recall that there are two main aspects of kernelized algorithms: (i) the solution is expressed as a linear combination of training examples, (ii) the algorithm relies only on inner products between data points rather than their explicit representation. We will show that these two aspects can be satisfied in k -means.

1. [Extra credit: 5 pts] Let z_{ij} be an indicator that is equal to 1 if the x_i is currently assigned to the j^{th} cluster and 0 otherwise ($1 \leq i \leq n$ and $1 \leq j \leq k$). Show that the j^{th} cluster center c_j can be updated as $\sum_{i=1}^n \alpha_{ij} x_i$. Specifically, show how α_{ij} can be computed given all z 's.

2. [Extra credit: 5 pts] Given two data points x_1 and x_2 , show that the square distance $\|x_1 - x_2\|^2$ can be computed using only (linear combinations of) inner products.

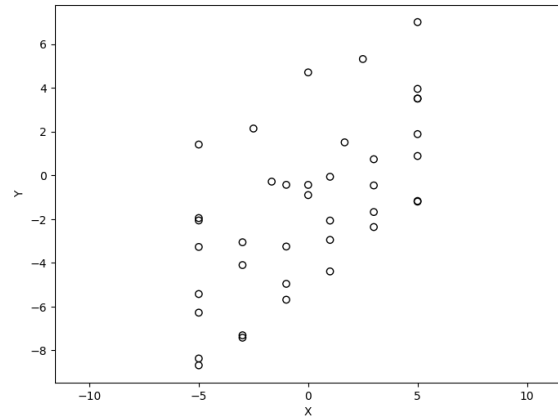
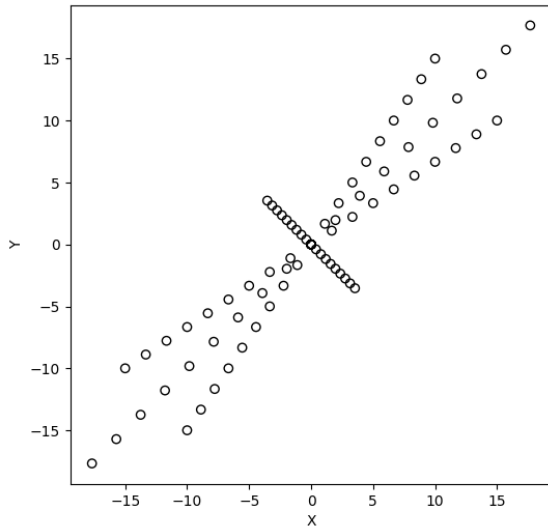
3. [Extra credit: 5 pts] Given the results of parts 2 and 3, show how to compute the square distance $\|x_i - c_j\|^2$ using only (linear combinations of) inner products between the data points $x_1, \dots, x_n$.

Note: This means that given a kernel K , we can run Lloyd's algorithm. We begin with some initial data points as centers and use the answer to part 2 to find the closest center for each data point, giving us the initial z_{ij} 's. We then repeatedly use the answer to part 3 to reassign the points to centers and update the z_{ij} 's.

4 PCA [20 pts + 10 Extra Credit]

4.1 Analysis

1. [10 pts] For each of the two data sets shown below, draw vectors that correspond to the first and second principal components that would be produced by Principal Components Analysis (PCA). For each diagram, label which vector is the first component, and which is the second.



2. [5 pts] Is it possible for you to draw a third principal component for the first of these datasets (in A)? Please do, or explain in one sentence why it is impossible.

3. [5 pts] PCA is typically performed before clustering high-dimensional data. For this procedure, what assumption do we make about the data's covariance structure in relation to the distances between clusters? Explain why this assumption holds or does not hold for the second dataset (in B). Be brief (2-3 sentences).

4.2 PCA two ways [EXTRA CREDIT: 10 pts]

Like the primal and dual problems for SVM, we can have two perspectives of PCA: maximizing the variance or minimizing the reconstruction error. We will find that both these views lead to the same optimization problem. This question uses a lot of linear algebra—make sure the sizes of each matrix and vector correctly match up!

4.2.1 [Extra credit: 5 pts] PCA: Maximizing the variance

Consider N data points $X_1, \dots, X_N \in \mathbb{R}^p$, such that the sample mean of X is zero: $\frac{1}{N} \sum_{i=1}^N X_i = 0$. Also consider projection vector $\mathbf{u} \in \mathbb{R}^p$, where $\|\mathbf{u}\|_2 = 1$. We would like to maximize the sample variance $\tilde{V}[\mathbf{u}^T X]$, which is the sample variance of X projected onto $\mathbf{u}$. The sample variance of N samples of a variable z is $\tilde{V}[z] = \frac{1}{N} \sum_{i=1}^N z_i^2$, where $\frac{1}{N} \sum_{i=1}^N z_i = 0$. We can write an optimization problem to maximize the sample variance:

$$\max_{\mathbf{u}} \quad \tilde{V}[\mathbf{u}^T X] \quad \text{s.t.} \quad \|\mathbf{u}\|_2 = 1 \quad (4.1)$$

Reformulate Eqn. 4.1 to the following optimization problem, and define the $p \times p$ matrix Σ :

$$\max_{\mathbf{u}} \quad \mathbf{u}^T \Sigma \mathbf{u} \quad \text{s.t.} \quad \|\mathbf{u}\|_2 = 1 \quad (4.2)$$

Show all work:

4.2.2 [Extra credit: 5 pts] PCA: Minimizing the reconstruction error

Consider the same variables as those defined in Question 4.1. Instead of maximizing the projected variance, we now seek to minimize the reconstruction error. In other words, we would like to minimize the difference between X and the reconstructed $\mathbf{u}\mathbf{u}^T X$. Note that $\mathbf{u}\mathbf{u}^T X$ first projects X onto $\mathbf{u}$, and then projects this scalar back into the p -dimensional space along the $\mathbf{u}$ axis. Minimizing the reconstruction error can be written as the following optimization problem:

$$\min_{\mathbf{u}} \quad \frac{1}{N} \sum_{i=1}^N \|X_i - \mathbf{u}\mathbf{u}^T X_i\|_2^2 \quad \text{s.t.} \quad \|\mathbf{u}\|_2 = 1 \quad (4.3)$$

Reformulate Eqn. 4.3 to the following optimization problem, with the same Σ as defined in Section 4.2.1.

$$\max_{\mathbf{u}} \quad \mathbf{u}^T \Sigma \mathbf{u} \quad \text{s.t.} \quad \|\mathbf{u}\|_2 = 1 \quad (4.4)$$

Show all work:

5 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment? **Solution Yes / No.**
(b) If you answered 'yes', give full details (e.g. "Jane Doe explained to me what is asked in Question 3.4")

Solution

2. (a) Did you give any help whatsoever to anyone in solving this assignment? **Solution Yes / No.**
(b) If you answered 'yes', give full details (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

Solution

3. (a) Did you find or come across code that implements any part of this assignment? **Solution Yes / No.**
(b) If you answered 'yes', give full details (book & page, URL & location within the page, etc.).

Solution